Git Commands

Easy to Advanced — Complete Reference Guide

Created by Shubham Soni

1. Basic / Daily Use

Command	Description
`git status`	See what files have changed in your working directory
`git add .`	Stage all changes for the next commit
`git add <file>`	Stage a specific file only
`git commit -m "msg"`	Commit staged changes with a message
`git commit --amend -m "msg"`	Edit the last commit message
`git push`	Push commits to the remote repository
`git pull`	Fetch and merge changes from remote
`git clone <url>`	Clone a remote repository locally
`git init`	Initialise a new Git repository

2. Branching

Command	Description
`git branch`	List all local branches
`git branch <name>`	Create a new branch
`git checkout <name>`	Switch to an existing branch
`git checkout -b <name>`	Create and switch to a new branch in one step
`git merge <branch>`	Merge a branch into the current branch
`git merge --no-ff <branch>`	Merge always creating a merge commit
`git branch -d <name>`	Delete a branch (safe — only if merged)
`git branch -D <name>`	Force delete an unmerged branch
`git branch -m <old> <new>`	Rename a branch

3. Inspection & History

Command	Description
`git log`	View full commit history
`git log --oneline`	Compact one-line commit history
`git log --oneline --graph --all`	Visual branch tree in the terminal
`git diff`	Show unstaged changes
`git diff --staged`	Show staged changes
`git diff <b1> <b2>`	Compare two branches
`git blame <file>`	See who last changed each line
`git show <commit>`	Show full details of a specific commit
`git shortlog -sn`	Commit count grouped by author

4. Undoing Things

Command	Description
`git restore <file>`	Discard unstaged changes in a file
`git restore --staged <file>`	Unstage a file without losing changes
`git reset HEAD~1`	Undo last commit, keep changes unstaged
`git reset --hard HEAD~1`	Undo last commit and discard all changes
`git reset --soft HEAD~1`	Undo last commit, keep changes staged
`git revert <commit>`	Create a new commit that undoes a previous one
`git clean -fd`	Remove all untracked files and directories
`git stash`	Temporarily save uncommitted changes
`git stash pop`	Restore the most recent stash
`git stash list`	List all saved stashes
`git stash drop`	Delete a specific stash
`git stash apply stash@{2}`	Apply a specific stash by index

5. Remote & Collaboration

Command	Description
`git fetch`	Download remote changes without merging
`git fetch --all`	Fetch from all configured remotes
`git remote -v`	List all remote connections
`git remote add origin <url>`	Add a new remote named origin
`git remote remove <name>`	Remove a remote connection
`git push origin <branch>`	Push a specific branch to remote
`git push -u origin <branch>`	Push and set tracking upstream
`git push --force`	Force push — overwrites remote (use with care)
`git push --force-with-lease`	Safer force push — fails if remote has new commits
`git pull --rebase`	Pull using rebase instead of merge

6. Advanced

Command	Description
`git rebase <branch>`	Reapply your commits on top of another branch
`git rebase -i HEAD~3`	Interactively edit, squash, or reorder last 3 commits
`git cherry-pick <commit>`	Apply a specific commit to the current branch
`git cherry-pick <c1>..<c2>`	Apply a range of commits
`git bisect start`	Start binary search to find a bug-introducing commit
`git bisect good <commit>`	Mark a commit as good during bisect
`git bisect bad <commit>`	Mark a commit as bad during bisect
`git reflog`	View history of all HEAD movements — recover lost commits
`git tag <name>`	Create a lightweight tag at current commit
`git tag -a v1.0 -m "msg"`	Create an annotated tag with message
`git push origin --tags`	Push all tags to remote
`git submodule add <url>`	Embed another repo inside your repo
`git worktree add <path> <branch>`	Check out a branch in a separate folder

7. Git Commands for Interview

Covering All Patterns

Merge vs Rebase

Command	Description
`git merge <branch>`	Joins histories — creates a merge commit, non-destructive
`git rebase <branch>`	Moves commits on top of another branch — clean linear history

💡 *Rule of thumb: use merge for public/shared branches, rebase for local cleanup before merging.*

Fast-forward vs No-fast-forward

Command	Description
`git merge <branch>`	Fast-forwards if possible — no merge commit created
`git merge --no-ff <branch>`	Always creates a merge commit even if fast-forward is possible

Reset vs Revert vs Restore

Command	Description
`git reset`	Moves HEAD — rewrites history (dangerous on shared branches)
`git revert`	Safe undo — creates a new commit, preserves history
`git restore`	Only affects working tree / staging — does not touch history

Soft vs Mixed vs Hard Reset

Command	Description
`git reset --soft HEAD~1`	Undo commit — keep changes staged
`git reset HEAD~1`	Undo commit — keep changes unstaged (mixed, default)
`git reset --hard HEAD~1`	Undo commit — discard all changes permanently

Git Commands — Easy to Advanced

Created by Shubham Soni

Stash vs Branch

Command	Description
`git stash`	Quick temporary save — not a commit, no branch context
`git checkout -b <name>`	Proper save with full history, context, and traceability

Detached HEAD

Command	Description
`git checkout <hash>`	Puts you in detached HEAD — not on any branch
`git checkout -b <name>`	Fix detached HEAD by creating a branch at current position

Squashing Commits

Command	Description
`git rebase -i HEAD~3`	Interactive rebase — mark commits as squash to combine them

💡 *Squash before merging a feature branch to keep the main branch history clean and readable.*

Conflict Resolution Flow

Command	Description
`git merge <branch>`	Step 1 — start merge, conflict occurs
`(edit file)`	Step 2 — open file and resolve <<<<<<< markers manually
`git add <file>`	Step 3 — stage the resolved file
`git commit`	Step 4 — complete the merge

Tracking & Upstream

Command	Description
`git push -u origin <branch>`	Set upstream so future git push works without arguments
`git branch -vv`	See tracking/upstream info for all local branches

Git Hooks

Command	Description
`pre-commit`	Run linting or tests before every commit
`commit-msg`	Enforce commit message format or conventions
`post-merge`	Auto-install dependencies after a pull/merge

Useful One-liners for Interviews

Command	Description
`git log --oneline --graph --all`	Visualise full branch history in the terminal
`git diff HEAD`	See all changes since the last commit
`git stash push -m "WIP: feature"`	Save a stash with a descriptive name
`git reflog`	Recover lost commits after an accidental hard reset
`git blame -L 10,20 <file>`	Show blame for a specific line range only

Created by Shubham Soni

linkedin.com/in/shubhamsoni-374a86175 • 210k+ Followers